

P3719R0

`std::is_vector_bool_reference<T>`

Published Proposal, 2025-05-20

This version:

<https://wg21.link/p3719>

Author:

[Yuxuan Chen](#)

Audience:

LEWG, SG18

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

- 1 Revision History**
- 2 Introduction**
 - 2.1 Motivation
 - 2.2 Existing Practices
 - 2.3 Proposed Solution: A Type Trait
- 3 Or Should We Change Template Argument Deduction Instead?**
- 4 Impact on the standard**
- 5 Proposed Wording Changes**

§ 1. Revision History

R0: Initial version

§ 2. Introduction

This paper proposes to add a new type trait `std::is_vector_bool_reference<T>` that exposes the previously exposition-only constant `is_vector_bool_reference` in `[vector.bool.pspc]/8` to `<type_traits>`. The trait `std::is_vector_bool_reference<T>::value` evaluates to true if and only if `T` denotes the type `vector<bool, Alloc>::reference` for some type `Alloc` and `vector<bool, Alloc>` is not a program-defined specialization.

§ 2.1. Motivation

This new trait is intended to help code like this

```
// Illustration only, library may contain many overloads for `read`
void read(bool& b);
void read(int& b);
void read(long& b);
void read(long long& b);
// ... many more
```

One who writes this code (who may be working with a serialization library like Apache Thrift) will soon realize that this needs to work with `std::vector<bool>`, in which the `bool`s aren't actual `bool` objects. Some may find tempting to add an overload that looks like this:

```
void read(std::_Bit_reference b);
```

However, this code isn't portable due to its dependency on `libstdc++`'s implementation details. This unfortunately occurs a lot in a massive code base that previously used `libstdc++`.

Another attempt to write this in standard C++ would look something like:

```
void read(std::vector<bool>::reference b);
```

This works fine for most cases and seemingly solved the problem with standard C++, until your function `read` runs into a reference to a `std::vector<bool, MyAllocator>` with a custom allocator.

This approach sometimes doesn't work. That is because `std::vector<bool, MyAllocator>::reference` and `std::vector<bool>::reference` may or may not be the same type. In fact, they are the same type in `libstdc++`, which is just `std::_Bit_reference` defined

[here](#). While in libc++ and MSVC STL, this is defined as a template whose argument may contain the vector type.

When someone runs into such an error, an intuitive but naive thought would be to make the allocator type in the vector parameterized.

```
template <typename Alloc>
void read(std::vector<bool, Alloc>::reference b);
```

However, compilers can't deduce `Alloc` here because it has no way to figure out which `vector` a `__bit_reference<vector<bool, Alloc>>` has come from. It's impractical for any user of this function to provide the allocator type either. This solution is still not ideal.

§ 2.2. Existing Practices

This is not the first time anyone has run into this problem. In fact, the `[vector.bool.pspc]/8` states that there's an exposition-only constant `is_vector_bool_reference` that is true if `T` denotes the type `vector<bool, Alloc>::reference` for some type `Alloc` and `vector<bool, Alloc>` is not a program-defined specialization. The only use of the constant is formatter in `[vector.bool.fmt]`.

libc++ implements the exposition-only constant through a check [here](#).

§ 2.3. Proposed Solution: A Type Trait

This paper is to propose that we just provide this as a type trait in standard C++ along the lines of:

```
template <typename T>
constexpr bool is_vector_bool_reference_v = /* ... */

template <class T> struct is_vector_bool_reference;
```

This [link](#) to compiler explorer provides some implementation experience for libstdc++, libc++, and MSVC STL. The implementation is straightforward.

Since `libc++` also shares the bit reference definition with `std::bitset<N>::reference`, inclusion of forward declaration of vector here is required, but otherwise it's the same.

Please note that `libc++'s` definition for `is-vector-bool-reference` also covers its standard noncompliant `__bit_const_reference` with `_LIBCPP_ABI_VERSION == 1`. Since this is already non-standard, whether it should return true for `__bit_const_reference` is left to libc++ maintainers to decide. It's not an issue for the motivating example, due to the expectation of a const ref would allow the `__bit_const_reference` to be converted to a bool.

With such a type trait, we can define the overload in the form of

```
template <typename BitReference>
    requires std::is_vector_bool_reference_v<BitReference>
void read(BitReference b);
```

And this works as expected.

§ 3. Or Should We Change Template Argument Deduction Instead?

An alternative would be introducing some core language change to make deducing the `Alloc` parameter for the function template

```
template <typename Alloc>
void read(std::vector<bool, Alloc>::reference b);
```

possible. However, this requires looking at all partial specializations of the class template, finding the dependent member, removing the specialization where this name doesn't exist or is not a type, then deduce the template parameter `Alloc` for the class based on the knowledge you have with the incoming type. Even then, deducing `Alloc` may still fail because some implementations of the `_Bit_reference` don't depend on `Alloc`. This approach does not immediately seem feasible. The library solution proposed in this paper should be the better solution.

§ 4. Impact on the standard

This paper proposes a library only change that shouldn't impact the core language. Since the standard already depends on the exposition-only constant `is-vector-bool-reference` we can make the formatter use this type trait to constrain the template specialization instead.

§ 5. Proposed Wording Changes

TODO: Make changes to

[meta.type.synop]

[meta.unary.prop]

[vector.bool.pspc]/8

[vector.bool.fmt]